

BACC: Budget-Aware Calibration and Control for Horizontal Autoscaling

Fan Liu

Guanqi Li

Behrooz Farkiani

Patrick Crowley

Washington University in St. Louis

St. Louis, USA

{fan.liu, guanqi, b.farkiani, pcrowley}@wustl.edu

ABSTRACT

Cloud services must continuously adapt replica counts to fluctuating demand while respecting fixed-period reliability budgets. Many horizontal autoscalers either react to instantaneous utilization or provision against a fixed predictive risk target. These policies do not explicitly account for how much of the period-level violation budget has already been consumed, so they can be overly conservative when the budget is healthy and insufficiently conservative when the budget is being depleted.

We present **BACC**, a model-agnostic framework for budget-aware horizontal autoscaling. BACC separates three concerns that are often entangled in prior systems: workload prediction, online uncertainty calibration, and budget-paced capacity control. It wraps an arbitrary forecaster with Adaptive Conformal Inference (ACI) to calibrate workload uncertainty online, then uses a proportional-integral controller to adjust provisioning aggressiveness based on the observed pace of budget consumption. This decomposition preserves a clean interface between forecasting and control: the predictor estimates demand, the conformal layer calibrates uncertainty, and the controller decides how conservative the provisioning policy should be at the current point in the budget window.

We instantiate BACC for CPU-threshold-based horizontal autoscaling in Kubernetes and evaluate it through trace-driven simulation and cluster replay experiments. This CPU-instantiated realization serves as an end-to-end use case for a broader fixed-period compliance framework over scalar thresholded signals. Across five Azure Functions traces, three compliance levels, and two forecasting backends, BACC tracks the requested violation target closely, achieving mean absolute compliance gaps of 0.44 and 0.42 percentage points with ARIMA and Chronos, respectively. The Kubernetes experiments further show that the same controller improves CPU-threshold compliance over native HPA under deployment effects such as measurement delay and replica readiness.

1 INTRODUCTION

Motivation. Cloud applications must continuously adapt resources to fluctuating demand [8, 10, 18]. Under-provisioning causes overload and service degradation; over-provisioning wastes capacity and inflates cost. Modern orchestration platforms such as Kubernetes [17] and Borg [7] make fine-grained autoscaling operationally feasible, but they do not remove the core control problem: deciding how much capacity to provision before a surge becomes visible at runtime.

This paper studies *horizontal* autoscaling, where the controller changes the number of service replicas. The operational objective is not merely to keep a signal below a threshold at every instant. In practice, operators manage reliability through fixed-period SLO *error budgets* [4]: the system may exceed a threshold at most an ϵ fraction of time over a window such as a day or month. For autoscaling, this induces a budget-allocation problem. Spending the budget too quickly early in the period risks later exhaustion; spending it too conservatively wastes resources. An ideal controller should therefore adapt its aggressiveness to the *remaining* budget, not only to the current utilization snapshot.

Existing horizontal autoscalers are poorly matched to that objective. Reactive controllers such as Kubernetes HPA scale from current CPU measurements, which means they respond only after a demand increase is already observable. Predictive controllers reduce that lag by forecasting ahead, but most still use a fixed safety margin, fixed percentile, or fixed chance constraint throughout execution. Recent SLO-oriented systems add richer telemetry, stronger models, or optimization [22, 24, 30, 36], yet they still primarily optimize instantaneous thresholds or static probabilistic targets rather than pace how the fixed-period budget is consumed over time.

Challenges. Two challenges follow. First, autoscaling decisions are subject to actuation delay: new replicas must be scheduled, initialized, and become ready before they can absorb load [9, 16, 22]. A purely reactive controller therefore cannot pre-position capacity for fast workload increases. Second, proactive control requires reasoning about forecast uncertainty under non-stationary workloads, where trend shifts, changing periodicity, and bursts can quickly invalidate a fixed margin [26]. The controller needs uncertainty calibration, but it also needs a separate policy for deciding how conservative to be *at the current point in the budget window*. Conflating these two roles makes systems hard to tune and hard to generalize across workloads.

Our approach. We address this mismatch with **BACC**, a model-agnostic autoscaling framework that separates three concerns that prior systems often entangle: demand estimation, uncertainty calibration, and budget-paced resource allocation. The prediction layer estimates future workload. An Adaptive Conformal Inference (ACI) layer [13, 32] calibrates forecast uncertainty online under drift. A budget controller then decides how aggressively to provision, based on the observed pace of fixed-period violation-budget consumption. This division of labor matters: calibration answers “how uncertain

is the forecast?”, whereas the controller answers “how much risk can be spent now?”.

We instantiate this design first for CPU-based horizontal autoscaling. BACC consumes workload forecasts from either neural or statistical backends, calibrates them online, and then adjusts the effective CPU operating point through a proportional–integral controller that reacts to budget surplus or deficit. This CPU threshold can be interpreted either as a direct operational objective in infrastructure-facing settings or as an operational proxy calibrated from a user-facing SLI for CPU-bound services. Choosing CPU for the first end-to-end instantiation is deliberate: it is directly measurable, directly actionable by standard horizontal autoscalers, and shared by the baselines in our comparison, so it lets us test the control architecture without introducing a service-specific latency model. Because budget awareness lives downstream of calibration, the same controller can be reused with different forecasting backends without redesigning the control logic. More generally, the same formulation may apply to other scalar operational signals such as queue length, request backlog, or latency, provided they admit online measurement and thresholded violations.

Evaluation scope. Section 5 studies BACC through trace-driven simulation and Kubernetes cluster experiments, comparing against representative reactive, percentile-based, and predictive baselines.

Contributions.

(1) Fixed-period error-budget pacing for horizontal autoscaling. We formulate replica autoscaling under a fixed-period violation budget on a scalar operational signal, making the pace of violation-budget consumption an explicit control objective rather than an after-the-fact metric.

(2) Budget-aware conformal autoscaling architecture. We propose BACC, which separates base workload prediction, online uncertainty calibration via ACI, and downstream budget-paced capacity control. This design keeps predictive calibration distinct from provisioning policy and supports multiple forecasting backends within the same control loop.

(3) End-to-end CPU-instantiated realization and evaluation. We instantiate BACC for CPU-based horizontal autoscaling in both a trace-driven simulator and a Kubernetes controller, and evaluate this first end-to-end realization on Azure Functions traces [26] against representative reactive, percentile-based, and predictive baselines.

Paper organization. Section 2 formulates the fixed-period budget objective for scalar operational threshold signals and states the cost-minimization objective. Section 3 reviews reactive and proactive autoscaling and related work on conformal prediction. Section 4 presents BACC and the end-to-end autoscaling pipeline, and Section 5 reports the evaluation results.

2 PROBLEM FORMULATION

We consider a horizontally scalable cloud application whose capacity can be adjusted by changing the number of replicas (e.g., pods, containers). Time is discrete and indexed by minutes $t \in \{1, \dots, T\}$. Let $x_t \in \mathbb{Z}_{\geq 1}$ denote the provisioned capacity at minute t (number of replicas), and let $y_t \in \mathbb{R}_{\geq 0}$ denote the workload intensity (e.g., request rate) at minute t .

Autoscaling decisions are made online at control epochs. Let h denote the control period (minutes). At a decision time $t \in \{h, 2h, \dots\}$, the controller observes the available history

$$\mathcal{H}_t \triangleq \{(y_i, c_i, x_i)\}_{i=1}^t, \quad (1)$$

and selects a replica count x_{t+h} as a function of $\mathcal{H}_t$. Between control epochs, capacity remains unchanged except when a previously issued decision takes effect.

BACC only requires a scalar operational indicator whose threshold crossings represent overload risk. In the most general form, let z_t denote such an indicator and define violations through $\mathbf{1}[z_t > \tau]$ when larger values are worse. This indicator may itself be user-facing (e.g., latency or queue delay), or it may be an operational signal derived from a user-facing target through calibration. In this paper we instantiate z_t as CPU utilization, because it is widely available to autoscalers, directly tied to provisioning decisions in Kubernetes-like systems, and often serves as a practical overload proxy for CPU-bound services or infrastructure-facing deployments. The simulator and controller implementation we evaluate are therefore CPU-instantiated end-to-end, even though the budget-allocation formulation itself is not CPU-specific.

The realized CPU utilization $c_t \in [0, 1]$ depends on both workload and capacity. We model this dependence abstractly as

$$c_t = g(y_t, x_t) + \xi_t, \quad (2)$$

where $g(\cdot)$ is an unknown system response function and ξ_t captures noise and unmodeled effects (e.g., contention). Our design in Section 4 instantiates $g(\cdot)$ with a lightweight workload–CPU model to translate workload forecasts into replica counts.

2.1 Operational Threshold Objective and Fixed-Period Budget

BACC solves the fixed-period budget-allocation problem for a scalar violation indicator. In this paper, we study the CPU-instantiated version and define the per-minute violation indicator as

$$v_t \triangleq \mathbf{1}[c_t > \tau]. \quad (3)$$

For a fixed evaluation period of T minutes and an allowed violation rate $\epsilon \in (0, 1)$, define the total violation budget and cumulative violation count as

$$B \triangleq T\epsilon, \quad V_t^{\text{cum}} \triangleq \sum_{i=1}^t v_i. \quad (4)$$

The fixed-period compliance requirement is

$$V_T^{\text{cum}} \leq B, \quad (5)$$

or equivalently, the realized violation rate $S_{\text{vr}} = V_T^{\text{cum}}/T \leq \epsilon$.

For online control, it is useful to compare the realized budget consumption against the nominal budget pace

$$B_t^{\text{ref}} \triangleq t\epsilon, \quad (6)$$

which represents how many violations would have been spent by time t under uniform budget consumption. BACC uses this prefix reference only for pacing decisions; the hard requirement remains the end-of-window constraint in Eq. (5).

This formulation mirrors common production SLO practice, where error budgets are typically defined over a fixed calendar period [4] (e.g., a calendar month with $\epsilon = 0.01$ for a 99% SLO). The

controller manages how the budget is spent over time: spending it too fast early risks exhausting it before the period ends, while spending it too conservatively wastes resources. Our evaluation uses $T = 2,880$ minutes (2 days) as the test period; in deployment, T corresponds to the operator-defined budget window. In a user-facing deployment, the operational threshold τ could be selected by profiling the highest signal level at which the desired external SLI remains acceptable. The same formulation therefore applies if c_t is replaced by another scalar signal such as queue length, backlog, or latency, provided the system defines an application-specific threshold and a violation direction.

2.2 Objective: Cost Minimization Under a Fixed-Period Budget

Let $\text{cost}(x_t)$ be the per-minute resource cost (e.g., replica-minutes). The autoscaling goal is to minimize cost while satisfying the fixed-period budget, subject to operational constraints:

$$\min_{\{x_t\}_{t=1}^T} \sum_{t=1}^T \text{cost}(x_t) \quad (7)$$

$$\text{s.t. } V_T^{\text{cum}} \leq B, \quad (8)$$

$$x_{\min} \leq x_t \leq x_{\max}, \quad \forall t, \quad (9)$$

$$|x_{t+h} - x_t| \leq \rho, \quad \forall t \in \{h, 2h, \dots\}, \quad (10)$$

where ρ is the maximum allowed change in replica count per control epoch.

Because workload and system dynamics are uncertain, the controller must make decisions using only $\mathcal{H}_t$ (and possibly a workload forecaster). Our goal is to design an online policy that minimizes resource cost while maintaining fixed-period compliance over the evaluation period.

3 RELATED WORK

Autoscaling for cloud services has been studied extensively [8, 10, 18]. For this paper, the relevant comparison space is *horizontal replica autoscaling* for cloud-native services, with Kubernetes as a deployment context but not the sole boundary of related work. Within that scope, the most relevant literature falls into three buckets: threshold-based reactive autoscaling, SLO-oriented predictive or application-aware resource managers, and conformal methods for online time-series calibration.

3.1 Reactive Autoscaling

Reactive autoscalers observe current or recent utilization and adjust resources after demand changes are visible. Kubernetes HPA [17] and cloud-provider equivalents [1, 14] scale replicas from thresholded CPU signals, while KEDA [15] extends this pattern to event-driven signals such as queue depth or consumer lag. The central limitation of this family is actuation delay: by the time high utilization is observed and a new replica becomes ready, violations may already have accumulated [9, 16, 22]. Production studies likewise report persistent under- and over-provisioning under purely reactive control [19, 27].

More advanced reactive systems improve diagnosis or reduce oscillation without changing that basic timing limitation. Google Autopilot [23] learns from historical execution data to recommend

vertical limits and replica counts, but it still addresses the same single-service provisioning problem and extrapolates from past demand rather than explicitly forecasting future surges. FIRM [22] detects and localizes ongoing SLO violations from distributed telemetry, and ATOM [12] optimizes allocations using an online queueing model of the current load; both react to observed degradation rather than provisioning ahead of it. SATA [21] is the closest SLO-aware extension of HPA to our setting: it explicitly selects a CPU threshold to satisfy an external performance SLO, but remains reactive, does not track an explicit error budget, and offers no distribution-free uncertainty guarantee.

Overall, reactive methods differ in sophistication but share two gaps relevant to our work: they cannot reliably pre-position capacity before fast demand surges, and they generally do not adapt conservatism to the remaining fixed-period error budget [4].

3.2 SLO-Oriented Predictive and Application-Aware Autoscaling

Proactive autoscalers forecast future conditions and provision in advance, thereby decoupling scaling decisions from actuation latency. Prior systems differ mainly in what they predict and how they treat uncertainty.

Several methods use machine learning or optimization to predict workload, violations, or resource configurations directly. Kraken [5] forecasts per-service demand for serverless DAGs, Seer [11] predicts impending QoS violations from distributed traces, and Sinan [34] learns inter-service performance dependencies to allocate resources across tiers. More recent systems push this line further. Erms [20] builds scaling models for shared microservices with heterogeneous SLA impact. Autothrottle [30] uses a bi-level controller that translates end-to-end latency SLO feedback into per-service CPU throttle targets. Erlang [24] searches over microservice configurations to minimize dollar cost while meeting latency targets. Aquatope [35] uses Bayesian models to manage uncertainty in multi-stage serverless workflows. These systems show that application-aware modeling can materially improve SLO-resource tradeoffs, but they typically target end-to-end latency, require service-specific structure or offline profiling, and do not use the remaining fixed-period error budget to modulate uncertainty conservativeness online.

Other approaches incorporate explicit uncertainty handling. Opt-Scaler [36] is the closest prior system on this axis. It combines workload prediction with MPC and a Gaussian chance-constraint model for keeping CPU under a target threshold in a horizontal scaling loop. This yields a useful formal guarantee only under the assumed residual distribution, and the conservativeness target remains fixed throughout execution. Very recent work such as AAPA [33] adds workload archotyping and heuristic uncertainty adjustments for serverless workloads, but it likewise does not expose an explicit budget-aware risk controller. In contrast, BACC combines distribution-free online calibration with a separate budget-control layer that allocates the remaining violation budget over time.

In summary, proactive methods address the timing gap of reactive autoscaling, but existing approaches either rely on predictive models whose calibration can degrade under workload shift or

adopt static uncertainty targets that ignore the remaining fixed-period error budget.

3.3 Conformal Prediction and Online Calibration

Conformal prediction [25, 29] provides a distribution-free way to wrap a forecaster with calibrated prediction sets. Standard split-conformal methods assume exchangeability and produce static prediction intervals, which limits their use for non-stationary time series. ACI [13] extends this idea to online settings by updating the target miscoverage level over time, and Zaffran et al. [32] further study adaptive conformal calibration for time series under shift.

Recent conformal forecasting work further expands this design space. Sequential Predictive Conformal Inference (SPCI) predicts future residual quantiles directly for non-exchangeable time series [31]. Hallberg Szabadváry [28] extends ACI to online multi-step forecasting with step-wise coverage guarantees, and other recent conformal forecasting methods study more structured multi-series settings [32]. These methods improve predictive calibration for sequential data, but they optimize predictive coverage itself rather than using coverage as a control variable tied to an error budget.

Our work builds on ACI but uses it differently from prior budget-aware interpretations. Rather than making the conformal target itself budget-dependent, BACC uses ACI as an online uncertainty-calibration layer whose target coverage is determined by the requested compliance level, and places budget awareness in a separate control layer that adjusts provisioning aggressiveness over time. To our knowledge, prior conformal methods and prior autoscaling systems do not combine online conformal calibration with an explicit fixed-period error-budget-aware control policy for horizontal replica autoscaling. This separation between predictive calibration and budget allocation is the key distinction from standard ACI, from newer conformal forecasters such as SPCI, and from proactive autoscalers with fixed safety margins or fixed probabilistic targets.

4 DESIGN

Figure 1 illustrates the end-to-end architecture of our autoscaler. The system operates as a closed-loop pipeline with four layers executed every h minutes (control epoch): (1) a *Prediction Layer* that produces a forecast of future workload, (2) an *ACI Calibration Layer* that calibrates forecast uncertainty online to the requested fixed-period compliance level, (3) a *Budget-Control Layer* that adjusts the effective CPU operating point according to observed violation-budget consumption, and (4) a *Scaling Layer* that translates the calibrated forecast and effective CPU target into a replica count. Feedback from observed CPU utilization and workload closes the loop, updating the ACI nonconformity scores, the budget controller state, and the online estimate of the CPU-model slope w_k . Algorithm 1 summarizes the complete control loop.

This layered split also clarifies scope. The BACC architecture is generic over scalar thresholded signals: prediction and online uncertainty calibration are signal-agnostic, while the downstream control and capacity-translation layer depends on the operational signal being managed. In this paper, we instantiate that final layer for CPU-based horizontal autoscaling. A queue-length, backlog, or latency-based realization would preserve the same architecture

while replacing only the signal-specific observation model and capacity-translation components.

4.1 Prediction Layer

At each control epoch k (corresponding to minute $t = kh$), the prediction module produces a base forecast $\hat{y}_{t+1:t+h}$ over the next h minutes. We use an point forecast rather than tying the raw prediction directly to the compliance target. The forecasting layer estimates demand, while uncertainty calibration and compliance-specific aggressiveness are handled downstream. The module is *model-agnostic*: any forecaster, whether neural or statistical, may be plugged in without modifying the downstream pipeline. In our evaluation we instantiate two representative backends:

- **Chronos** [2, 3]: A pre-trained foundation model for time series forecasting. Given a rolling context window of past workload observations, Chronos exposes a central forecast for each future step, which we use as the point prediction.
- **ARIMA** [6]: A classical statistical model that produces point forecasts. We use the point forecast directly as the raw prediction.

Both backends expose the same interface: given the current minute t and a horizon h , they return a vector $\hat{y}_{t+1:t+h} \in \mathbb{R}_{\geq 0}^h$. This base prediction is then passed to the ACI layer for conformal adjustment.

4.2 ACI Calibration Layer

Standard ACI [13] maintains an adaptive miscoverage level α_t and a set of *nonconformity scores* to construct prediction intervals with distribution-free coverage guarantees. In BACC, ACI is used strictly as the uncertainty-calibration layer. Its target coverage is determined by the requested compliance level, but it is *not* directly modulated by the remaining violation budget. This separation avoids entangling forecast calibration with downstream resource-allocation policy.

4.2.1 Conformal Adjustment and ACI Update. Let $\alpha_{\text{base}} = 1 - q$ be the target miscoverage implied by the requested compliance level. At each control epoch, the ACI layer transforms the base prediction $\hat{y}_{t+j}$ for each future minute $j \in \{1, \dots, h\}$ into an adjusted prediction:

$$\tilde{y}_{t+j} = \hat{y}_{t+j} + \hat{Q}_{1-\alpha_t}, \quad (11)$$

where $\hat{Q}_{1-\alpha_t}$ is the $(1 - \alpha_t)$ -quantile of the rolling set of nonconformity scores $\{e_i\}$. The nonconformity score at each minute is the signed residual $e_i = y_i - \hat{y}_i$, measuring how much the actual workload exceeded the adjusted prediction.

When the actual workload y_m is observed at global minute $m = t + j$ (where $j \in \{1, \dots, h\}$), we adapt α_m via the clipped ACI update rule [13]:

$$\alpha_{m+1} = \text{clip}(\alpha_m + \gamma (\alpha_{\text{base}} - \mathbf{1}[y_m > \tilde{y}_m]), \alpha_{\min}, \alpha_{\max}), \quad (12)$$

where $\gamma > 0$ is the learning rate and $[\alpha_{\min}, \alpha_{\max}]$ clips α_m to a valid range. The update fires once per minute, producing h sequential updates to α across each length- h epoch. When a violation occurs ($y_m > \tilde{y}_m$), the indicator is 1 and α_m decreases, making the next prediction more conservative (larger conformal correction). When no violation occurs, α_m increases, reducing the correction. In BACC,

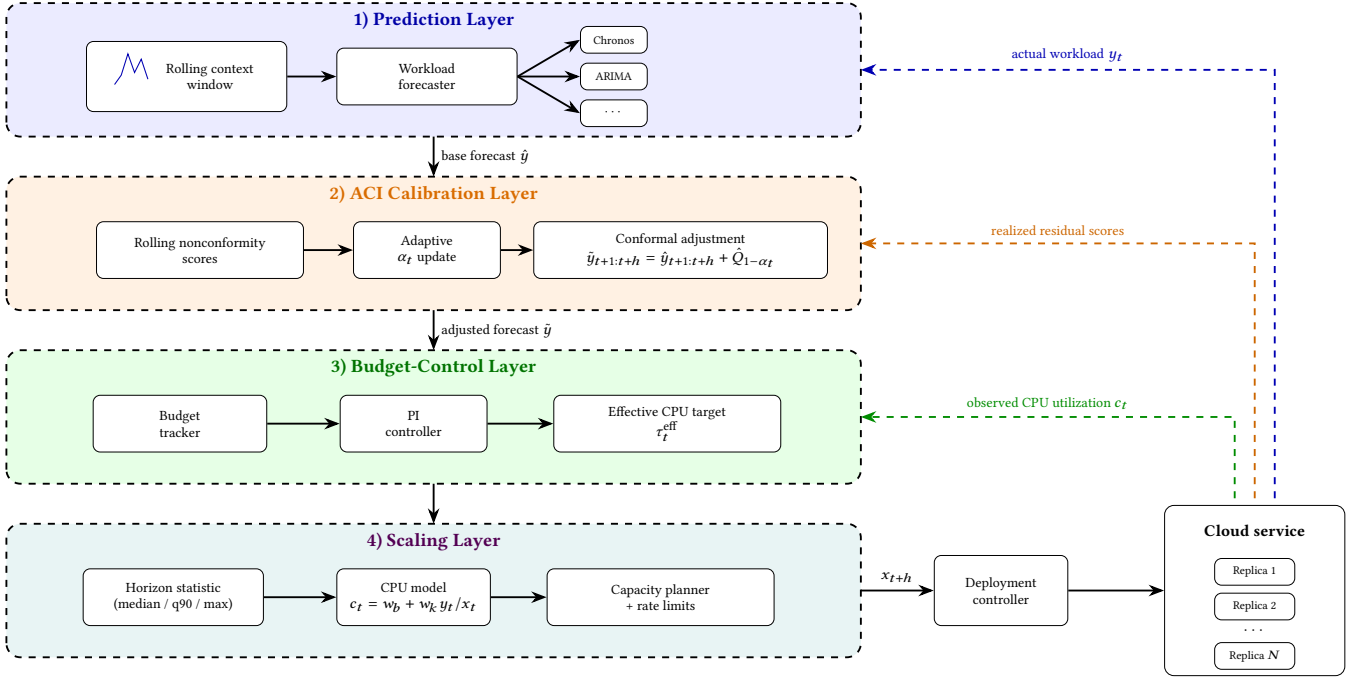


Figure 1: System overview of the proposed budget-aware autoscaler. Solid arrows denote the forward decision path; dashed arrows denote feedback signals from observed CPU and workload.

ACI therefore serves as the uncertainty-calibration layer, while the budget controller separately decides how aggressively the calibrated forecast should be translated into capacity.

4.3 Budget-Control and Scaling Layers

The budget-control and scaling module translates the adjusted workload forecast $\tilde{y}_{t+1:t+h}$ into a concrete replica count x_{t+h} . This stage is the signal-specific part of the current system. In our implementation it targets CPU utilization; a queue-length or latency instantiation would replace the signal model and capacity translation while leaving the upstream BACC controller unchanged. In the current simulator and Kubernetes controller, budget awareness acts directly on the CPU operating point via τ_t^{eff} rather than modifying the conformal target. This keeps the prediction and calibration layers model-agnostic while locating signal-specific control policy at the capacity-planning stage. When CPU is used only as an operational proxy, τ may be interpreted as a calibrated threshold chosen so that keeping CPU below it is expected to preserve an external SLI.

4.3.1 Budget Tracker. The budget layer monitors how quickly the fixed-period violation budget is being consumed in CPU space. Let

$$\epsilon = 1 - q \quad (13)$$

be the allowed violation rate implied by the requested compliance level q . We additionally allow a nonnegative budget margin $\delta_b \in [0, \epsilon)$, which lets the controller track a slightly stricter internal target

$$\epsilon_b = \epsilon - \delta_b. \quad (14)$$

Setting $\delta_b = 0$ recovers the exact violation budget, while $\delta_b > 0$ adds a configurable safety margin. Let

$$\hat{v}_t = \frac{V_t^{\text{cum}}}{t} \quad (15)$$

be the observed cumulative violation rate up to minute t , where $V_t^{\text{cum}} = \sum_{i=1}^t v_i$ and $v_i = 1[c_i > \tau]$ is the per-minute CPU-threshold violation indicator from Section 2. The controller compares realized spending against the nominal pace in Section 2 through the tracking error

$$e_t = \hat{v}_t - \epsilon_b. \quad (16)$$

If $e_t > 0$, violations are accumulating faster than allowed and the controller should tighten. If $e_t < 0$, the system is spending the budget more slowly than required and can relax.

4.3.2 PI Budget Controller. We use a proportional-integral (PI) controller on the effective CPU operating point. Minute-level violations v_t are recorded continuously, but in the released simulator and Kubernetes controller the PI update is evaluated only at control epochs ($t \bmod h = 0$), using the cumulative violation history observed up to that decision time. At each such epoch, the integral state is updated as

$$I_t = \text{clip}(I_{t-1} + e_t, -I_{\text{max}}, I_{\text{max}}), \quad (17)$$

and the control action is

$$u_t = K_P e_t + K_I I_t, \quad (18)$$

where K_P and K_I are the proportional and integral gains, respectively. The resulting effective CPU target is

$$\tau_t^{\text{eff}} = \text{clip}(\tau - u_t, \tau_{\text{min}}, \tau_{\text{max}}). \quad (19)$$

When the system is overspending its violation budget ($e_t > 0$), the controller lowers τ_t^{eff} , forcing a more conservative replica choice. When the system is under-spending its budget ($e_t < 0$), the controller raises τ_t^{eff} , allowing higher utilization and lower resource usage. The proportional term reacts to the current deviation; the integral term corrects persistent bias over time.

4.3.3 CPU Utilization Model. We instantiate the capacity planner with a lightweight linear workload-CPU model, using a formulation closest to OptScaler [36]:

$$c_t = w_b + w_k \cdot \frac{y_t}{x_t}, \quad (20)$$

where w_b is the baseline (idle) CPU overhead, w_k is the per-unit workload cost, and y_t/x_t is the per-replica workload. The parameters w_b and w_k can be estimated offline by regressing historical CPU observations against per-replica request rates, and w_k can be updated online as new request and CPU data arrive during deployment.

4.3.4 Capacity Planner. Given the selected adjusted forecast $\tilde{y}_{t+h}^*$ over the control horizon and the budget-adjusted CPU target τ_t^{eff} , the capacity planner determines the target replica count as follows:

- (1) **Find replica count under the effective CPU target:** Find the smallest x^* such that the predicted CPU utilization stays within τ_t^{eff} :

$$x^* = \min \left\{ x \in \mathbb{Z}_{\geq 1} \mid w_b + w_k \cdot \frac{\tilde{y}_{t+h}^*}{x} \leq \tau_t^{\text{eff}} \right\}. \quad (21)$$

Here $\tilde{y}_{t+h}^*$ is the scalar capacity target derived from the forecast horizon (e.g., max or a fixed percentile statistic).

- (2) **Enforce constraints:** Clip to capacity bounds and apply rate limits:

$$x_{t+h} = \text{clip} \left(x^*, \max(x_{\min}, x_t - \rho), \min(x_{\max}, x_t + \rho) \right),$$

where ρ is the maximum scaling change per interval.

Guarantee scope. BACC inherits the calibration role of ACI only at the workload-forecast layer: the conformal adjustment adapts online to recent workload prediction errors and targets the requested miscoverage level under the usual assumptions and limitations of adaptive conformal methods for non-stationary sequences. The end-to-end CPU-threshold objective, however, also depends on the workload-to-CPU translation, actuation delay, measurement lag, and replica-readiness dynamics. We therefore do not claim a distribution-free guarantee that $S_{\text{or}} \leq 1 - q$ for arbitrary deployments. Instead, the budget controller is an online feedback policy that uses observed CPU-threshold violations to pace the remaining fixed-period budget. The compliance results evaluated in this paper are empirical: BACC is tested on held-out traces and Kubernetes replay runs to measure how closely the closed-loop system tracks the requested CPU-threshold violation rate.

5 EVALUATION

Our evaluation consists of two parts: (1) a trace-driven simulation that enables controlled, reproducible comparison across diverse workload patterns and fixed-period compliance levels, and (2) a

Algorithm 1: BACC: Budget-Aware Calibration and Control for Horizontal Autoscaling

Input : Workload trace $\{y_t\}$, compliance level q , budget margin δ_b , CPU threshold τ , control interval h , learning rate γ , PI gains K_P, K_I , nonconformity score window W_s

Output : Capacity decisions $\{x_t\}$

- 1 Train forecaster on historical trace;
- 2 $\alpha_0 \leftarrow 1 - q$; // Initialize miscoverage
- 3 $\epsilon_b \leftarrow (1 - q) - \delta_b$; // Internal budget target
- 4 $\mathcal{S} \leftarrow \emptyset$; // Nonconformity scores
- 5 $I_0 \leftarrow 0$; // PI controller integral state
- 6 **for** $t = 1, 2, \dots, T$ **do**
- 7 Observe workload y_t and CPU c_t ;
- 8 **if** pending ACI update for minute t **then**
- 9 $e_t \leftarrow y_t - \tilde{y}_t$; // Nonconformity score
- 10 Update rolling window $\mathcal{S}$ with e_t ;
- 11 $\alpha_{t+1} \leftarrow \text{clip}(\alpha_t + \gamma((1 - q) - 1[y_t > \tilde{y}_t]), \alpha_{\min}, \alpha_{\max})$; // ACI update
- 12 $v_t \leftarrow 1[c_t > \tau]$; // CPU-threshold violation
- 13 Update V_t^{cum} and $\hat{v}_t \leftarrow V_t^{\text{cum}}/t$;
- 14 **if** $t \bmod h = 0$; // Control epoch
- 15 **then**
- 16 $e_t^{\text{budget}} \leftarrow \hat{v}_t - \epsilon_b$; // Budget tracking error at decision time
- 17 $I_t \leftarrow \text{clip}(I_{t-1} + e_t^{\text{budget}}, -I_{\max}, I_{\max})$;
- 18 $u_t \leftarrow K_P e_t^{\text{budget}} + K_I I_t$;
- 19 $\tau_t^{\text{eff}} \leftarrow \text{clip}(\tau - u_t, \tau_{\min}, \tau_{\max})$;
- 20 $\hat{y}_{t+1:t+h} \leftarrow \text{Forecaster}(t, h)$;
- 21 $\hat{Q}_{1-\alpha_t} \leftarrow \text{Quantile}(\mathcal{S}, 1 - \alpha_t)$; // Compute once per epoch
- 22 **for** $j = 1, \dots, h$ **do**
- 23 $\tilde{y}_{t+j} \leftarrow \hat{y}_{t+j} + \hat{Q}_{1-\alpha_t}$;
- 24 $\tilde{y}_{t+h}^* \leftarrow \text{HorizonStatistic}(\hat{y}_{t+1:t+h})$;
- 25 $x^* \leftarrow \min\{x \in \mathbb{Z}_{\geq 1} \mid w_b + w_k \tilde{y}_{t+h}^*/x \leq \tau_t^{\text{eff}}\}$;
- 26 $x_{t+h} \leftarrow \text{clip}(x^*, \max(x_{\min}, x_t - \rho), \min(x_{\max}, x_t + \rho))$;

Kubernetes cluster deployment that validates the framework under deployment effects including container startup delays and resource contention. The simulation provides the main comparative evaluation across methods, traces, and compliance levels; the Kubernetes experiments complement it by validating BACC in a real control loop.

5.1 Experimental Setup

Datasets. We evaluate our framework using five representative workload traces (A–E) derived from the Azure Functions Trace 2019 [26], spanning 14 days (July 15–28, 2019) at one-minute granularity (20,160 data points each). Prediction difficulty increases from Trace A to Trace E, as summarized in Table 1. Trace A is smooth and highly periodic. Trace B is bursty and sparse (14.5% zero-involution minutes) but retains daily periodicity. Trace C is high-volume with strong short-term but no daily structure. Trace D

Table 1: Summary statistics of workload traces A–E. P99/P50: burstiness ratio; AC(1h), AC(1d): autocorrelation at 1-hour and 1-day lags; Entropy: normalized spectral entropy of the power spectral density; Zero%: percentage of minutes with zero invocations. Prediction difficulty increases from A to E, reflecting higher variability, burstiness, and lower autocorrelation.

Trace	Mean	Median	P99/P50	AC(1h)	AC(1d)	Entropy	Zero%
A	2868	3018	1.53	0.92	0.89	0.10	0.00
B	2170	1448	6.99	0.83	0.78	0.23	14.51
C	9920	9573	2.85	0.83	0.06	0.37	0.04
D	6734	4469	6.38	0.50	0.44	0.55	0.64
E	1464	985	10.61	0.10	0.20	0.84	0.00

is right-skewed with bursty, unpredictable demand. Trace E is the most challenging, with near-zero autocorrelation and high spectral entropy approaching white noise.

Simulation environment. All experiments use a discrete-time simulation at one-minute resolution with the linear CPU model described in Section 4.3.3 ($w_b = 0.10$, $w_k = 0.005$). These are the values used in our simulation instantiation: w_b is the idle CPU fraction, and w_k is derived from estimated per-request CPU cost and mean request duration ($w_k = \text{cpu_per_req} \times \text{duration_s} / (60 \times \text{cores})$). The first 12 days of each trace serve as training data for the forecasting model; the remaining 2 days (2,880 minutes) are used for evaluation. Proactive autoscalers (BACC and OptScaler) make scaling decisions every $h = 5$ minutes. Autopilot also uses a 5-minute scaling interval, following the original paper [23]. HPA uses $h = 1$ minute, the finest granularity permitted by the simulation, to approximate its real-world behavior (Kubernetes HPA evaluates every 15 seconds by default [17]). All methods use a CPU target of $\tau = 0.5$. We choose CPU for this first end-to-end study because it is the common operational signal directly available to all compared autoscalers and to the Kubernetes controller, enabling a like-for-like systems comparison without introducing application-specific latency instrumentation.

Metrics. We report three metrics for each experiment:

- S_{or} (%): the fixed-period CPU-threshold violation rate over the 2 test days, i.e., the percentage of minutes where $c_t > \tau$.
- V_{sum} : the cumulative violation magnitude, $\sum_{t: c_t > \tau} (c_t - \tau)$.
- R_{avg} : the average number of provisioned replicas over the test period.

In this CPU-instantiated evaluation, an autoscaler satisfies the requested compliance level if $S_{or} \leq (1 - q) \times 100\%$, where q is the target fraction of minutes kept below the CPU threshold. For brevity, we denote $q \in \{0.90, 0.95, 0.99\}$ by P90, P95, and P99, respectively. Although our main objective is defined on CPU, we treat it as an *operational saturation metric* rather than a user-facing SLA.

Baselines. We compare BACC against three baseline approaches in simulation:

- **Kubernetes HPA** [17]: The default Horizontal Pod Autoscaler (HPA), a reactive mechanism that scales based on

current CPU utilization against a target threshold. We implement HPA following the official Kubernetes documentation with default parameters.

- **Google Autopilot** [23]: A reactive autoscaler that uses an exponentially-weighted histogram of recent CPU usage to recommend resource limits at the S -th percentile. In our common evaluation setup, we implement the horizontal recommendation logic described in the original paper; we do not reproduce its vertical resource sizing or GKE-specific scheduling components, which are not publicly available. We evaluate it at $S \in \{0.90, 0.95, 0.99\}$, matching the CPU-threshold compliance level under test.
- **OptScaler** [36]: A proactive autoscaler that uses MPC with chance constraints. At each control epoch, OptScaler solves an optimization problem over a D -interval horizon to maximize CPU utilization subject to a probabilistic guarantee $\Pr[c_t \leq \tau] \geq \alpha$, where α is the chance constraint parameter. The constraint is enforced via the closed-form bound $x_d \geq m_d / (\tau - w_b - \Phi^{-1}(\alpha)\sigma_b)$, where m_d aggregates predicted workload with CPU model uncertainty. In our common evaluation setup, we evaluate OptScaler at $\alpha \in \{0.90, 0.95, 0.99\}$, matching each requested compliance level q , and use $D = 11$ intervals (55-minute horizon with $h = 5$). An online linear regression (OLR) module updates the CPU model parameters during simulation. We adopt $D = 11$ and enable OLR because the original OptScaler paper reports this as the stronger configuration; using it avoids handicapping the baseline in our comparison. Since OptScaler’s original hybrid Fourier-sequence predictor is not publicly available, we supply it with the same out-of-box ARIMA or Chronos point forecasts used by BACC.

HPA is purely reactive and unaware of the requested compliance level, producing the same scaling decisions regardless of q . Autopilot and OptScaler expose tunable parameters (S and α , respectively) that can be set to match a desired compliance level, but neither adapts these parameters dynamically based on the remaining violation budget during the simulation.

Prediction backends. We provide both BACC and OptScaler with forecasts from the same prediction backend. Both controllers consume the backend’s point forecast. OptScaler estimates uncertainty through its parametric noise model, whereas BACC applies an external ACI layer on top of the same point prediction. This setup isolates the autoscaling control logic from prediction quality, though it should be interpreted as a controlled comparison of controller behavior rather than a full reproduction of every baseline system.

We evaluate with two backends that differ in model family and inductive bias: a deep pre-trained foundation model versus a classical statistical forecaster. This pairing tests the model-agnostic nature of BACC across fundamentally different prediction mechanisms while keeping the downstream control stack unchanged.

Chronos [3] is a foundation model for time series forecasting developed by Amazon. It is pre-trained on a large corpus of both real-world and synthetic time series data using a language-modeling objective over quantized token sequences. In our framework, Chronos operates in a zero-shot setting: it receives the full observed history up to the current minute as context and returns a central forecast for

the prediction horizon, requiring no task-specific fine-tuning [2, 3]. We use this central forecast as the backend point prediction supplied to the autoscaler.

ARIMA [6] is a classical statistical forecasting model that combines autoregressive (AR), differencing (I), and moving average (MA) components. In the $\text{ARIMA}(p, d, q)$ formulation, p denotes the order of the autoregressive term (capturing dependence on the previous p observations), d denotes the order of differencing (applied to achieve stationarity), and q denotes the order of the moving average term (modeling the effect of the previous q forecast errors). Our implementation uses a rolling ARIMA forecaster with a 180-minute context window and automatic order selection via `auto_arima` on the recent context. For efficiency, the selected order is cached and refreshed periodically rather than re-optimized at every prediction step.

Rather than fitting ARIMA on the full history, we use a rolling context window of the most recent 180 minutes at each prediction step; the model is refit on this local window and forecasts $h = 5$ steps ahead, keeping fitting fast and ensuring the model tracks recent dynamics rather than stale patterns. ARIMA provides only a point forecast in our implementation. For BACC, any uncertainty inflation is handled downstream by the external ACI layer rather than by the forecasting backend itself. This setup reflects a lightweight, off-the-shelf statistical forecasting pipeline when paired with the BACC controller.

This separation between forecasting and control is intentional. Instead of relying on a workload-specific predictor that must be re-trained or fine-tuned whenever the workload distribution changes, BACC treats the forecaster as a pluggable backend and performs compliance correction online through calibration and budget feedback. As new data arrive, the controller adapts its uncertainty adjustment and violation-budget pacing without requiring a new model-training cycle. This makes the framework better suited to operational settings where traces drift over time and retraining a specialized model for each new workload is costly or impractical.

Autoscaler configuration. Table 2 summarizes the parameters for all methods. Our framework is evaluated with two prediction backends, Chronos [3] and ARIMA [6], across three fixed-period compliance levels: P90, P95, and P99. Autopilot is evaluated at the matching percentile statistic S , and OptScaler sets the chance constraint parameter α to the corresponding compliance level. HPA uses fixed default parameters and is unaware of the requested compliance level.

5.2 Overall Simulation Results

Table 3 and Figure 2 summarize the simulation results across all five traces and three fixed-period compliance levels.

Reactive HPA is limited by actuation delay. HPA records S_{or} between 15.6% and 19.0% on all five traces, exceeding the loosest P90 target in this workload-driven setup. This behavior is consistent with its reactive design: the controller scales from current CPU measurements, so upward demand shifts are observed only after higher utilization has already appeared. Its lower resource use should therefore be interpreted together with the higher observed violation rate, rather than as a uniformly better resource-compliance tradeoff.

Table 2: Evaluation settings and autoscaler configurations. Common settings apply to all methods; autoscaler-specific controller parameters are listed below.

Parameter	Description	Value
Common Evaluation Settings		
q	Requested compliance level	{0.90, 0.95, 0.99}
τ	CPU utilization target	0.50
$x_{\min}$	Minimum replicas	1
$x_{\max}$	Maximum replicas	1,000
ρ	Max. replicas change per interval	100
Training	Training period (days)	12
Test	Test period (days)	2
BACC		
h	Scaling interval (min)	5
γ	ACI learning rate	0.08
W_s	Nonconformity score window (min)	720
$[\alpha_{\min}, \alpha_{\max}]$	Miscoverage clipping bounds	$[0.5(1 - q), \min(0.5, 6(1 - q))]$
δ_b	Budget margin on target violation rate	0.005
$[\tau_{\min}, \tau_{\max}]$	Effective CPU target bounds	$[0.30, 0.70]$
K_P	PI proportional gain	0.40
K_I	PI integral gain	0.05
$I_{\max}$	Integral-state clipping bound	5.0
OptScaler		
h	Scaling interval (min)	5
D	MPC horizon (intervals)	11
α	Chance constraint level	{0.90, 0.95, 0.99}
OLR	Online parameter learning	Enabled
η	OLR learning rate	2×10^{-4}
K8s HPA		
h	Scaling interval (min)	1
Tolerance	Scaling dead-zone	0.10
Stabilization	Downscale stabilization (min)	5
Google Autopilot		
h	Scaling interval (min)	5
T	Recommendation horizon (min)	4,320
Statistic	Horizon statistic	{P90, P95, P99}
Tolerance	Scaling dead-zone	0.05
Stabilization	Downscale stabilization (min)	30
$t_{1/2}$	Slow-decay half-life (min)	60

Autopilot is effective on smooth demand but less aligned with fixed-period budgets. Under our common evaluation setup, Autopilot satisfies 2 of 15 trace-compliance combinations: Trace A at P90 ($S_{\text{or}} = 4.8\%$) and Trace A at P95 ($S_{\text{or}} = 4.7\%$). On the burstier traces, its observed violation rates are higher than the requested fixed-period targets, with $S_{\text{or}} = 9.7\text{--}16.8\%$ on Trace B, $3.4\text{--}12.1\%$ on Trace D, and $6.0\text{--}16.4\%$ on Trace E. This is consistent with the role Autopilot plays in our comparison: it extrapolates from a histogram of recent CPU usage, whereas the target studied here requires anticipating workload changes and pacing a remaining violation budget. On Trace C at P99, for example, Autopilot records $S_{\text{or}} = 9.6\%$ while provisioning 257.3 replicas on average.

OptScaler is sensitive to fixed-risk calibration under bursty traces. OptScaler improves over HPA on Trace B, but its fixed chance-constraint setting does not consistently match the requested fixed-period targets across the full matrix. With the ARIMA backend it satisfies none of the 15 combinations; with Chronos it satisfies 2 of 15, both on Trace B (P90 and P95). On the harder traces C–E, the fixed Gaussian chance constraint appears sensitive to the forecast-error distribution: on Trace E, for example, OptScaler records $S_{\text{or}} =$

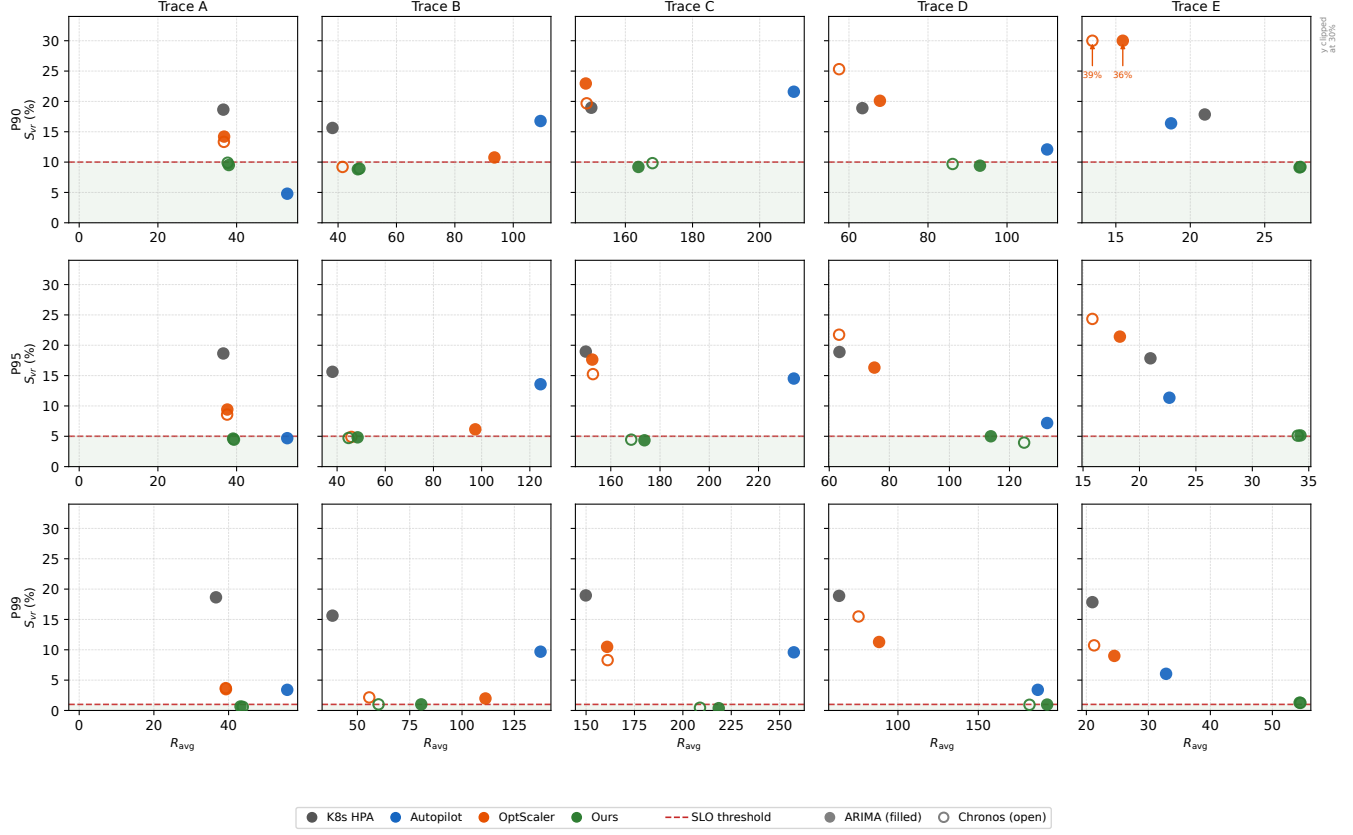


Figure 2: Resource-compliance tradeoff (S_{or} vs R_{avg}) across fixed-period compliance levels and traces. Dashed line = target threshold; shaded region = compliant zone.

35.5%, 21.4%, 9.0% with ARIMA and 39.1%, 24.3%, 10.7% with Chronos across P90/P95/P99. In our controlled setup, OptScaler uses the same forecast backends as BACC, so some of this gap is attributable to forecast quality. We do not fine-tune these predictors on the evaluation traces: Chronos is used zero-shot, and ARIMA uses the same automatic rolling configuration throughout. The remaining pattern suggests that a fixed parametric risk bound can be difficult to align with heavy-tailed forecast errors and a finite-period violation budget.

BACC most consistently stays near the requested target. Across the 30 backend-trace-compliance combinations, BACC exactly satisfies 24. The remaining 6 misses are all small and are confined to the two hardest settings: Trace B at P99 ($S_{or} = 1.01\%$ for both backends) and Trace E at P95/P99 (5.14%/1.28% with ARIMA and 5.10%/1.25% with Chronos). This is the central comparative result: adding budget awareness helps the controller track the requested fixed-period target across a wider range of traces and compliance levels. The resource-compliance tradeoff is also favorable in several cases. On Trace B at P95, BACC uses 48.5 replicas with ARIMA and 44.8 with Chronos, versus 124.4 for Autopilot, while reducing S_{or} from 13.6% to 4.83% and 4.72%.

Violation severity follows the same pattern. The cumulative violation magnitude V_{sum} captures how much overload mass accumulates beyond the threshold, not just how often the threshold is crossed. BACC also improves this metric substantially on the difficult traces. On Trace C at P95, BACC reduces V_{sum} to 5.73 (ARIMA) and 5.99 (Chronos), versus 37.01 for Autopilot and 26.28/19.76 for OptScaler. On Trace D at P95, BACC records 27.59 (ARIMA) and 19.68 (Chronos), compared with 33.81 for Autopilot and 90.94/132.65 for OptScaler. When BACC provisions more replicas than a less protective configuration, the extra capacity is therefore associated with a materially smaller overload budget overrun rather than merely adding slack.

The same controller adapts across compliance levels without retuning. A stricter compliance level naturally drives BACC to provision more conservatively. On Trace B with ARIMA, R_{avg} rises from 46.8 at P90 to 48.5 at P95 and 80.5 at P99, while S_{or} falls from 8.82% to 4.83% and then to 1.01%. Across traces, the observed violation rate stays close to the allowed violation rate, which is consistent with the design of a budget-paced controller: spend slack when it exists, then tighten automatically as the budget becomes scarce.

The remaining gap appears at the workload-to-CPU translation layer. The conformal layer calibrates workload forecasts, but the

Table 3: CPU-threshold violation rate S_{or} (%) across methods, traces, and fixed-period compliance levels. Cell colors are based on the displayed one-decimal value relative to the target threshold; greener is better.

Method	S_A	S_B	S_C	S_D	S_E
P90 (threshold $\leq 10\%$)					
K8s HPA	18.6	15.6	19.0	18.9	17.8
Autopilot	4.8	16.8	21.6	12.1	16.4
OptScaler (ARIMA)	14.2	10.8	23.0	20.1	35.5
OptScaler (Chronos)	13.3	9.2	19.7	25.3	39.1
Ours (ARIMA)	9.5	8.8	9.2	9.4	9.1
Ours (Chronos)	9.9	8.9	9.8	9.7	9.2
P95 (threshold $\leq 5\%$)					
K8s HPA	18.6	15.6	19.0	18.9	17.8
Autopilot	4.7	13.6	14.5	7.2	11.4
OptScaler (ARIMA)	9.4	6.1	17.6	16.3	21.4
OptScaler (Chronos)	8.6	4.9	15.2	21.7	24.3
Ours (ARIMA)	4.6	4.8	4.3	5.0	5.1
Ours (Chronos)	4.4	4.7	4.4	4.0	5.1
P99 (threshold $\leq 1\%$)					
K8s HPA	18.6	15.6	19.0	18.9	17.8
Autopilot	3.4	9.7	9.6	3.4	6.0
OptScaler (ARIMA)	3.5	2.0	10.5	11.3	9.0
OptScaler (Chronos)	3.7	2.2	8.3	15.5	10.7
Ours (ARIMA)	0.7	1.0	0.4	1.0	1.3
Ours (Chronos)	0.6	1.0	0.5	0.9	1.2

budgeted CPU-threshold objective is enforced only after those forecasts are mapped through the linear CPU model. The residual misses all occur where that mapping leaves very little slack: Trace B at P99 and Trace E at P95/P99. On Trace E at P99, both backends overshoot by only 0.25–0.28 percentage points. This suggests that the dominant remaining error is not instability in the budget controller itself, but small systematic underestimation after translation into CPU space. Extending the conformal layer to calibrate directly on the CPU signal is therefore a natural next step.

5.3 Ablation Study

To isolate the contribution of each design component, we compare three modes of BACC:

- **Raw:** No conformal adjustment — the scaling decision is derived directly from the base point forecast, with no ACI correction.
- **ACI:** Standard ACI with a fixed target miscoverage $\alpha_{\text{base}} = 1 - q$ and no budget-control layer. The ACI state still adapts from recent forecast errors, but the CPU operating point remains fixed at τ rather than responding to the remaining violation budget.
- **BACC (proposed):** compliance-targeted ACI together with the budget controller on the CPU operating point.

Table 4 reports the mean absolute compliance gap, $|S_{or} - \text{target}|$, averaged over all traces and compliance levels for each backend.

Table 4: Compact ablation of the three BACC layers. We report mean absolute SLO gap $|S_{or} - \text{target}|$ (%) averaged over all traces and SLO targets. Lower is better.

Method	ARIMA	Chronos
	Mean $ S_{or} - \text{target} $	Mean $ S_{or} - \text{target} $
Raw	38.56	37.67
ACI	10.44	10.38
BACC	0.44	0.42

Only the full budget-aware stack tracks the target closely. The ablation result is unambiguous. With ARIMA, the mean absolute compliance gap falls from 38.56 for Raw to 10.44 for ACI and then to 0.44 for BACC. With Chronos, it falls from 37.67 to 10.38 and then to 0.42. Thus ACI alone removes roughly 72–73% of the target gap, but the budget-aware controller removes a further 96% of the remaining error.

Calibration alone is not enough. The Raw mode has no online correction, so its base point forecasts are not calibrated to the actual compliance target. Adding ACI improves calibration, but it still uses a fixed target risk level and a fixed CPU operating point, so it cannot decide when to spend or conserve the remaining violation budget. The large residual gap of about 10 percentage points shows that online calibration without budget pacing still fails to match the fixed-period objective we care about.

Budget awareness, not just better forecasting, is what closes the loop. After deduplicating repeated Chronos rows in the raw result dump, BACC satisfies 12 of 15 trace–compliance combinations for each backend; the three misses per backend are the same borderline cases already discussed above: Trace B at P99 and Trace E at P95/P99. By contrast, neither Raw nor ACI satisfies any combination. The ablation therefore supports the paper’s main claim directly: conformal calibration is necessary, but budget-aware control is the component that turns calibrated forecasts into target-tracking autoscaling behavior.

5.4 Sensitivity Analysis

We next test how sensitive BACC is to the main controller parameters: the budget margin δ_b , the PI gains K_P/K_I , and the ACI score-window length W_s . We run a one-factor-at-a-time sweep using the ARIMA backend at P95 on the three harder traces used in the robustness study (Traces B, D, and E). Table 5 reports the mean violation rate, mean absolute compliance gap, and mean replica count after deduplicating repeated smoke-test rows in the raw sensitivity output.

The default margin is close to the best tradeoff. Increasing δ_b makes the controller more conservative by reserving more violation budget. With no margin, BACC averages $S_{or} = 5.38\%$, slightly above the P95 target; with $\delta_b = 0.010$, it averages 4.58% but uses more replicas. The default $\delta_b = 0.005$ gives the smallest mean gap in this sweep (0.10 percentage points) while keeping resource use close to the lower-margin settings.

Table 5: BACC sensitivity analysis with the ARIMA backend at P95, averaged over Traces B, D, and E. The target violation rate is 5%; lower mean gap is better.

Sweep	Setting	Mean S_{or} (%)	Mean $ S_{or} - 5 $	Mean R_{avg}
δ_b	0	5.38	0.38	64.69
δ_b	0.0025	5.17	0.17	65.15
δ_b	0.005	4.99	0.10	65.52
δ_b	0.010	4.58	0.42	67.15
K_P/K_I	0.4/0.0	10.71	5.71	48.79
K_P/K_I	0.2/0.025	5.20	0.57	62.30
K_P/K_I	0.4/0.05	4.99	0.10	65.52
K_P/K_I	0.8/0.10	4.78	0.47	72.02
W_s	360 min	4.84	0.16	66.78
W_s	720 min	4.99	0.10	65.52
W_s	1440 min	5.17	0.17	63.75

Integral feedback is important for budget tracking. Removing the integral term causes persistent under-provisioning: the mean violation rate rises to 10.71%, even though the controller uses fewer replicas. Half-strength gains improve tracking but remain looser than the default. Doubling both gains makes the controller more conservative and more expensive, reducing the mean violation rate to 4.78% but increasing R_{avg} from 65.52 to 72.02. This supports the default gains as a balanced operating point rather than a brittle single setting.

The ACI window is not a fragile parameter. Changing W_s from 360 to 1440 minutes keeps the mean compliance gap within 0.07 percentage points of the default setting. The shorter window reacts faster but is slightly more conservative, while the longer window is less expensive but slightly overshoots the target. Across the tested range, the controller remains near the requested fixed-period violation rate.

5.5 Model Agnosticism

BACC behaves similarly under both forecasting backends. The final controller’s mean absolute compliance gap is 0.44 percentage points with ARIMA and 0.42 with Chronos, and both backends satisfy 12 of 15 trace-compliance combinations exactly. On many traces the differences are small: on Trace D at P99, both backends achieve $S_{or} = 0.94\%$, with $R_{avg} = 191.7$ for ARIMA and 181.8 for Chronos; on Trace A at P95, both remain within about 0.2 percentage points of each other in both S_{or} and R_{avg} .

The main backend differences are workload-specific and modest. On Trace E, both backends are close to the target: at P95/P99, ARIMA records $S_{or} = 5.14\%/1.28\%$, compared with 5.10%/1.25% for Chronos. Chronos is somewhat more efficient on the more periodic Trace B, where at P95 it uses 44.8 replicas versus 48.5 for ARIMA while achieving a nearly identical violation rate (4.72% vs. 4.83%). This is consistent with the broader point that the controller behavior remains stable across both backends, and the residual differences are attributable to forecast quality on specific traces rather than to any backend-specific retuning in BACC.

The two backends also differ in computational cost. ARIMA is refit online at each 5-minute epoch on a 180-minute rolling window, requiring milliseconds per step and negligible memory overhead. Chronos is a transformer-based foundation model with significantly higher inference latency and GPU memory requirements. For latency-sensitive control loops or resource-constrained deployments, ARIMA provides a practical and effective default; Chronos is preferable when workload exhibits complex multi-scale patterns that benefit from its pre-trained representations.

5.6 Kubernetes Cluster Experiments

To validate our simulation findings in a deployment setting, we deploy BACC as a custom Kubernetes controller and replay workload traces against a containerized microservice. We compare BACC against Kubernetes native HPA, which is the production-standard autoscaler available on any Kubernetes distribution.

Setup. We evaluate BACC on a local Kubernetes cluster created with kind. The cluster uses a single control-plane node with max-pods=250. The target application is a CPU-bound microservice, `cpu-nginx`, deployed in the default namespace. The service is initially launched with one replica and exposes HTTP endpoints for the frontend and synthetic work generation. Each pod requests 100m CPU and 128Mi memory, with limits of 100m CPU and 256Mi memory to maintain the invocations’ requested resources. For the HPA baseline, we use the standard Kubernetes autoscaling/v2 controller with a target CPU utilization of 50%, zero scale-up stabilization, and 300s downscale stabilization.

Workload traces are replayed using Fortio from a dedicated in-cluster replay pod. Replay follows the same two-day test split used in simulation: the first 12 days of each trace are skipped (17280 rows), and the following 2 days are issued online as Poisson arrivals. For BACC, we use the same controller parameters as in simulation, run the ARIMA backend with the same control horizon ($h = 5$ minutes), and evaluate P95 and P99 compliance levels. Each experiment runs for 2 days. The Kubernetes controller uses the same point-forecast backend structure as the simulator: ARIMA supplies the base point prediction, and BACC’s external ACI layer performs the uncertainty calibration online. Cluster resource measurements are collected through Kubernetes metrics-server configured at 15s resolution. We report the same three evaluation metrics used in simulation: CPU-threshold violation rate S_{or} , cumulative violation magnitude V_{sum} , and average provisioned replicas R_{avg} .

Results. Table 6 summarizes the Kubernetes cluster results on Traces B and E. We report the CPU-threshold violation rate S_{or} , cumulative violation magnitude V_{sum} , and average provisioned replicas R_{avg} for native HPA and BACC under different compliance configurations. Since HPA is purely reactive and does not explicitly optimize for different compliance levels, we report a single HPA result per trace.

The populated Kubernetes results show the same qualitative pattern as the simulation: BACC substantially improves CPU-threshold protection relative to reactive HPA, while requiring only moderate additional capacity. On Trace B at P95, BACC reduces the violation rate from 19.45% under HPA to 4.61%, satisfying the P95 target, while R_{avg} increases modestly from 34.69 to 39.36. It also reduces

Table 6: Kubernetes cluster results comparing BACC and native HPA on Traces B and E. Lower is better for S_{or} , V_{sum} , and R_{avg} .

Trace	Method	SLO	S_{or} (%)	V_{sum}	R_{avg}
B	HPA	–	19.45	19.51	34.69
B	BACC	P95	4.61	10.67	39.36
B	BACC	P99	0.87	3.07	45.49
E	HPA	–	12.13	19.80	17.23
E	BACC	P95	4.44	4.85	25.07
E	BACC	P99	1.25	1.15	33.27

V_{sum} from 19.51 to 10.67. Tightening BACC to P99 on the same trace reduces S_{or} further to 0.87% and V_{sum} to 3.07, with $R_{avg} = 45.49$. On Trace E, BACC reduces S_{or} from 12.13% under HPA to 4.44% at P95 and 1.25% at P99, with V_{sum} decreasing from 19.80 to 4.85 and 1.15, respectively. These results indicate that the budget-aware controller can preserve or closely track the requested CPU-threshold compliance even in the deployment setting, where startup delay, measurement lag, and controller granularity make the control problem harder than in simulation.

We still treat the Kubernetes study as a deployment-side validation rather than as the primary aggregate comparison, which remains the fully populated simulation matrix above. The cluster experiments are narrower in scope, but they confirm that the simulation trend carries over to a real Kubernetes control loop on both a bursty periodic trace (Trace B) and a harder low-autocorrelation trace (Trace E).

5.7 Limitations and Future Work

This paper evaluates BACC as a CPU-instantiated realization of a broader fixed-period budget-control framework. CPU is a useful operational saturation signal because it is directly observable by all compared autoscalers and directly actionable in Kubernetes, but it is not itself a user-facing latency SLO. Extending the same control law to latency, queue length, or backlog requires replacing the signal-specific measurement and capacity-translation layer and validating the resulting threshold against application-level behavior.

The simulator uses a lightweight linear workload-CPU model, so the simulation results primarily isolate controller behavior under controlled capacity translation rather than all sources of production uncertainty. The Kubernetes experiments address some deployment effects, including measurement delay and replica readiness, but they run on a local kind cluster with a CPU-bound microservice and should be interpreted as deployment-side validation rather than a large-scale production study.

Finally, our Autopilot and OptScaler implementations follow the public descriptions available to us but do not reproduce proprietary or unavailable components, such as Autopilot’s GKE-specific scheduling stack or OptScaler’s original forecasting backend. We therefore interpret the comparisons as controlled evaluations of representative reactive, percentile-based, and fixed-risk predictive policies under a common workload and CPU-control interface. Future work includes evaluating BACC on user-facing latency and backlog signals, learning operational thresholds from external SLIs, and validating the controller in larger multi-node deployments.

6 CONCLUSION

We presented BACC, a budget-aware autoscaling framework that separates workload prediction, online uncertainty calibration, and budget-paced capacity control. The central design choice is to keep conformal calibration and provisioning policy distinct: ACI calibrates forecast uncertainty online, while a downstream PI controller decides how aggressively to spend the remaining fixed-period violation budget. This separation keeps the controller model-agnostic and makes the policy portable across forecasting backends.

Our evaluation indicates that this decomposition improves the compliance–resource tradeoff over reactive and fixed-risk baselines while working with both ARIMA and Chronos backends. More importantly, the paper frames fixed-period SLO management as a control problem in its own right: the autoscaler should not only satisfy a threshold target in aggregate, but should pace how quickly the violation budget is consumed over time. The CPU-based system in this paper is an end-to-end instantiation of that broader idea.

ACKNOWLEDGMENTS

This work was supported by NSF CNS Award 2213672.

REFERENCES

- [1] Amazon Web Services. 2026. *Auto Scaling Documentation*. AWS. <https://docs.aws.amazon.com/autoscaling/>. Accessed: 2026-02-17.
- [2] Abdul Fatir Ansari, Oleksandr Shchur, Jari Kiken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, Mononito Goswami, Shubham Kapoor, Danielle C. Maddix, Pablo Gueron, Tony Hu, Junming Yin, Nick Erickson, Prateek Mutalik Desai, Hao Wang, Huzefa Rangwala, George Karypis, Yuyang Wang, and Michael Bohlke-Schneider. 2025. Chronos-2: From Univariate to Universal Forecasting. *arXiv preprint arXiv:2510.15821* (2025). <https://arxiv.org/abs/2510.15821>
- [3] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Syndar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. 2024. Chronos: Learning the Language of Time Series. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=gerNCVqqTR>
- [4] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site reliability engineering: how Google runs production systems*. O’Reilly Media, Inc.
- [5] Vivek M Bhasi, Jashwant Raj Gunasekaran, Prashanth Thinakaran, Cyan Subhra Mishra, Mahmut Taylan Kandemir, and Chita Das. 2021. Kraken: Adaptive container provisioning for deploying dynamic dags in serverless platforms. In *Proceedings of the ACM Symposium on Cloud Computing*. 153–167.
- [6] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. 2015. *Time series analysis: forecasting and control*. John Wiley & Sons.
- [7] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. 2016. Borg, omega, and kubernetes. *Commun. ACM* 59, 5 (2016), 50–57.
- [8] Tao Chen, Rami Bahsoon, and Xin Yao. 2018. A survey and taxonomy of self-aware and self-adaptive cloud autoscaling systems. *ACM Computing Surveys (CSUR)* 51, 3 (2018), 1–40.
- [9] Valentin Flunkert, Quentin Rebjock, Joel Castellon, Laurent Callot, and Tim Januschowski. 2020. A simple and effective predictive resource scaling heuristic for large-scale cloud applications. *arXiv preprint arXiv:2008.01215* (2020).
- [10] Guilherme Galante, Luis Carlos Erpen De Bona, Antonio Roberto Murry, Bruno Schulze, and Rodrigo da Rosa Righi. 2016. An analysis of public clouds elasticity in the execution of scientific applications: a survey. *Journal of Grid Computing* 14, 2 (2016), 193–216.
- [11] Yu Gan, Yanqi Zhang, Kelvin Hu, Dailun Cheng, Yuan He, Meghna Pancholi, and Christina Delimitrou. 2019. Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 19–33.
- [12] Alim Ul Gias, Giuliano Casale, and Murray Woodside. 2019. ATOM: Model-driven autoscaling for microservices. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 1994–2004.
- [13] Isaac Gibbs and Emmanuel Candes. 2021. Adaptive conformal inference under distribution shift. *Advances in Neural Information Processing Systems* 34 (2021), 1660–1672.

- [14] Google Cloud Docs. 2026. *Load Balancing and Autoscaling*. Google Cloud. <https://docs.cloud.google.com/compute/docs/load-balancing-and-autoscaling> Accessed: 2026-02-17.
- [15] KEDA Project. 2024. *KEDA: Kubernetes Event-Driven Autoscaling*. CNCF. <https://keda.sh/docs/> Accessed: 2026-02-17.
- [16] Nane Kratzke and Peter-Christian Quint. 2017. Understanding cloud-native applications after 10 years of cloud computing-a systematic mapping study. *Journal of Systems and Software* 126 (2017), 1–16.
- [17] Kubernetes Documentation. 2026. *Horizontal Pod Autoscaling*. Kubernetes. <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/> Accessed: 2026-02-17.
- [18] Tania Lorido-Botran, Jose Miguel-Alonso, and Jose A Lozano. 2014. A review of auto-scaling techniques for elastic applications in cloud environments. *Journal of grid computing* 12, 4 (2014), 559–592.
- [19] Chengzhi Lu, Kejiang Ye, Guoyao Xu, Cheng-Zhong Xu, and Tongxin Bai. 2017. Imbalance in the cloud: An analysis on alibaba cluster trace. In *2017 IEEE International Conference on Big Data (Big Data)*. IEEE, 2884–2892.
- [20] Shutian Luo, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Jian He, Guodong Yang, and Chengzhong Xu. 2022. Erms: Efficient resource management for shared microservices with SLA guarantees. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 62–77.
- [21] Olesia Pozdniakova, Dalius Mažeika, and Aurimas Cholomskis. 2024. SLA-adaptive threshold adjustment for a Kubernetes horizontal pod autoscaler. *Electronics* 13, 7 (2024), 1242.
- [22] Haoran Qiu, Subho S Banerjee, Saurabh Jha, Zbigniew T Kalbarczyk, and Ravishanker K Iyer. 2020. {FIRM}: An intelligent fine-grained resource management framework for {SLO-Oriented} microservices. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)*. 805–825.
- [23] Krzysztof Rządca, Paweł Findeisen, Jacek Swiderski, Przemysław Zych, Przemysław Broniek, Jarek Kusmerek, Paweł Nowak, Beata Strack, Piotr Witusowski, Steven Hand, et al. 2020. Autopilot: workload autoscaling at google. In *proceedings of the fifteenth european conference on computer systems*. 1–16.
- [24] Vighnesh Sachidananda and Anirudh Sivaraman. 2024. Erlang: Application-aware autoscaling for cloud microservices. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 888–923.
- [25] Glenn Shafer and Vladimir Vovk. 2008. A tutorial on conformal prediction. *Journal of Machine Learning Research* 9 (2008).
- [26] Mohammad Shahradd, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX annual technical conference (USENIX ATC 20)*. 205–218.
- [27] Xiaoyang Sun, Chunming Hu, Renyu Yang, Peter Garraghan, Tianyu Wo, Jie Xu, Jianyong Zhu, and Chao Li. 2018. Rose: Cluster resource scheduling via speculative over-subscription. In *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 949–960.
- [28] Johan Hallberg Szabadváry. 2024. Adaptive conformal inference for multi-step ahead time-series forecasting online. *arXiv preprint arXiv:2409.14792* (2024).
- [29] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. 2005. *Algorithmic learning in a random world*. Springer.
- [30] Zibo Wang, Pinghe Li, Chieh-Jan Mike Liang, Feng Wu, and Francis Y. Yan. 2024. Autothrottle: A Practical Bi-Level Approach to Resource Management for SLO-Targeted Microservices. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 149–165. <https://www.usenix.org/conference/nsdi24/presentation/wang-zibo>
- [31] Chen Xu and Yao Xie. 2023. Sequential Predictive Conformal Inference for Time Series. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Vol. 202. PMLR, 38707–38727. <https://proceedings.mlr.press/v202/xu23r.html>
- [32] Margaux Zaffran, Aymeric Dieuleveut, Olivier Féron, Yannig Goude, and Julie Josse. 2022. Adaptive conformal predictions for time series. In *International Conference on Machine Learning*. PMLR, 25834–25866.
- [33] Guilin Zhang, Srinivas Vippagunta, Raghavendra Nandagopal, Suchitra Raman, Jeff Xu, Marcus Pfeiffer, Shreesankar Chatterjee, Ziqi Tan, Wulan Guo, and Hailong Jiang. 2025. AAPA: An Archetype-Aware Predictive Autoscaler with Uncertainty Quantification for Serverless Workloads on Kubernetes. *arXiv preprint arXiv:2507.05653* (2025).
- [34] Yanqi Zhang, Weizhe Hua, Zhuangzhuang Zhou, G Edward Suh, and Christina Delimitrou. 2021. Sinan: ML-based and QoS-aware resource management for cloud microservices. In *Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems*. 167–181.
- [35] Zhuangzhuang Zhou, Yanqi Zhang, and Christina Delimitrou. 2022. Aquatope: Qos-and-uncertainty-aware resource management for multi-stage serverless workflows. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 1–14.
- [36] Ding Zou, Wei Lu, Zhibo Zhu, Xingyu Lu, Jun Zhou, Xiaojin Wang, Kangyu Liu, Kefan Wang, Renen Sun, and Haiqing Wang. 2024. OptScaler: A Collaborative Framework for Robust Autoscaling in the Cloud. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4090–4103. <https://doi.org/10.14778/3685800.3685829>